



Universität
Zürich^{UZH}

ULB

UNIVERSITÉ LIBRE DE BRUXELLES

TCT Measurements and Analysis on Trench-Isolated Low-Gain Avalanche Detectors (Ti-LGADs)

Internship at UZH

Mathias Beghuin

Under the supervision of

Pr.Dr. Anna Macchiolo

Pr.Dr. Pascal Vanlaer

Under the effective supervision of

Dr. Leena Diehl

Academic year
2025 - 2026

Contents

1	Introduction	1
1.1	Silicon Detectors in High-Energy Physics	1
1.1.1	Time resolution	2
1.1.2	Low-Gain Avalanche Detectors (LGADs)	2
1.1.3	Ti-LGADs	2
1.1.4	Efficiency	2
1.1.5	Time resolution	2
2	The Transient Current Technique	3
2.1	TCT measurments	3
2.2	The TCT setup at UZH	3
3	Analysis program for TCT measurements	4
3.1	Structure of the GitHub repository	5
3.2	Description of the Ti-LGAD analysis program	5
3.2.1	main.py	6
3.2.2	config.py	6
3.2.3	plot_2D_maps.py	7
3.2.4	amplitude.py	9
3.2.5	charge_collection.py	13
3.2.6	timing.py	15
3.2.7	interpad.py	17
3.2.8	data_manager.py	20
3.2.9	plot_saved_results.py	24
4	Results	25
5	Conclusion	26
	Bibliography	27

Chapter 1

Introduction

High-energy physics experiments at colliders rely heavily on precise tracking detectors to reconstruct the tracks of charged particles produced in high energy collisions. The LHC experiments at CERN constitute the most relevant example of this dependence on tracking devices. The High Luminosity LHC (HL-LHC) and future colliders, will further increase the demands on tracking capabilities, due to higher collisions rates and radiation levels.

Low-Gain Avalanche Detectors (LGADs) have emerged as a promising technology in terms of temporal resolution, but also in terms of radiation hardness. Their ability to provide precise timing information, in addition to spatial measurements, makes them particularly suitable for 4D tracking applications in high-energy physics experiments. Unfortunately, these detectors suffer from a limited spatial resolution, known as *the fill-factor problem*. Further technological development has led to the introduction of Trench-Isolated LGADs (Ti-LGADs), which aim to address this limitation by incorporating trench structures to improve spatial resolution while maintaining the timing performance of traditional LGADs. This work focuses on the characterization of the performance of Ti-LGADs samples from the AIDAinnova production campaign, produced at FBK with different trench designs. This was done using a Transient Current Technique (TCT) setup with an infrared laser, allowing to study the impact of the trench structures on the detectors' performance. A relevant part of this work was dedicated to the improvement, optimisation and development of a python program used to analyse the TCT data at the Physik-Institut at University of Zurich (UZH).

1.1 Silicon Detectors in High-Energy Physics

Silicon is a widely used material in high-energy physics experiments due to its semiconductor properties and its radiation hardness. This section provides a brief description of the operating principle of silicon radiation sensors, as well as a description of LGADs and Ti-LGADs technologies.

Figure 1.1 shows a simplified schematic representation of the cross section of a planar silicon sensor. It represents a p-in-n (PIN) diode, composed of a p type substrate with a n-type implant on the top surface and a p++ implant on the back side, forming a p-n junction. The sensor is typically reverse biased until full depletion is reached, meaning that the electric field extends throughout the entire active thickness of the sensor.

When a high energy charged particle traverses the sensor, the impinging particle transfers a fraction of its kinetic energy to the silicon atoms through successive electromagnetic interactions, and leading to electron hole pair creation. The electric field present in the sensor causes the generated electrons and holes to drift towards their respective electrodes (holes to the p++

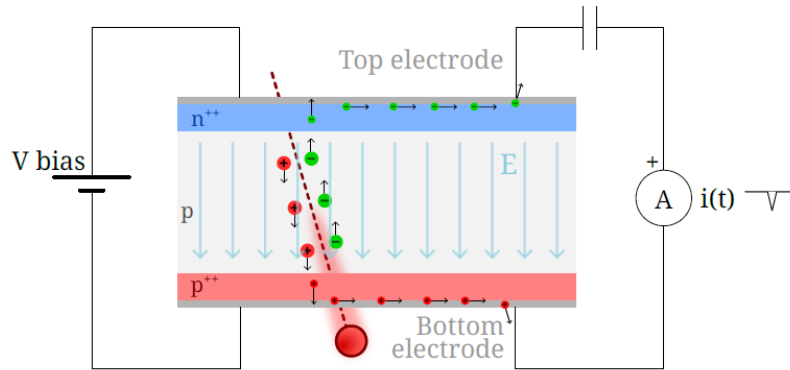


Figure 1.1: Simplified schematic representation of the cross section of a planar silicon sensor, taken from [1]

back implant and electrons to the n++ top implant). If a low impedance AC circuit is connected to the sensor electrodes, the movement of the charge carriers induces a current in the external circuit, which can then be measured.

1.1.1 Time resolution

The time resolution (σ_t) of a detector is a measure of its ability to accurately determine the time of arrival of a particle. Time measurements

1.1.2 Low-Gain Avalanche Detectors (LGADs)

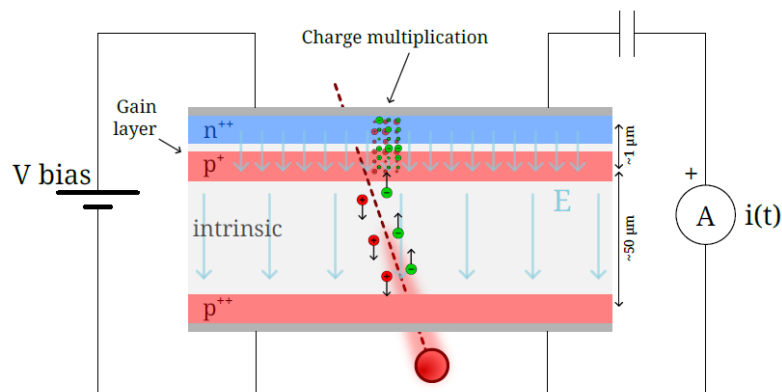


Figure 1.2: Simplified schematic representation of the cross section of a planar silicon LGAD sensor, taken from [1]

1.1.3 Ti-LGADs

1.1.4 Efficiency

1.1.5 Time resolution

Chapter 2

The Transient Current Technique

2.1 TCT measurments

2.2 The TCT setup at UZH

Chapter 3

Analysis program for TCT measurements

This section describes the development of a Python program for the data analysis of the TCT setup at UZH, starting from an initial script written by The current state of the project can be found on the [following GitHub repository](#). The program was developed and optimized to analyze TCT measurements on Ti-LGADs, but a version of the initial code has also been adapted by Leena Diehl to analyze TCT data on CASSIA sensors. Furthermore, some parts of the code can be reused for the analysis of TCT measurements on other types of sensors.

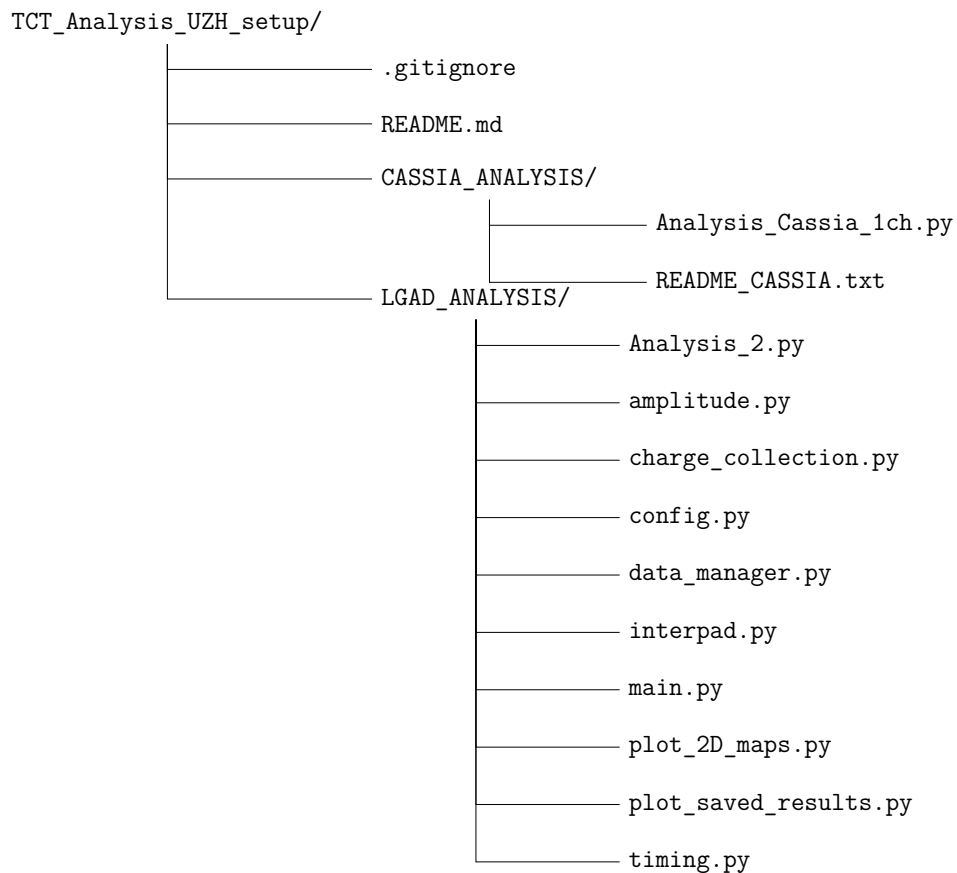


Figure 3.1: Structure of the GitHub repository for the TCT analysis program developed at UZH.

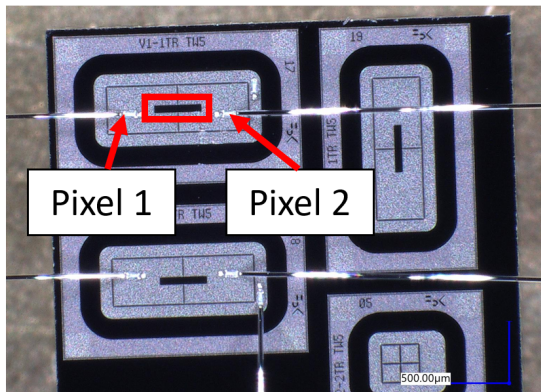
3.1 Structure of the GitHub repository

The structure of the GitHub repository is shown in the diagram shown on Figure 3.1. The repository contains two main folders: CASSIA_ANALYSIS and LGAD_ANALYSIS, which contain the analysis scripts for CASSIA sensors and Ti-LGADs, respectively. The main contribution to this project is the development of the LGAD_ANALYSIS folder, which includes various Python scripts for different aspects of the TCT data analysis.

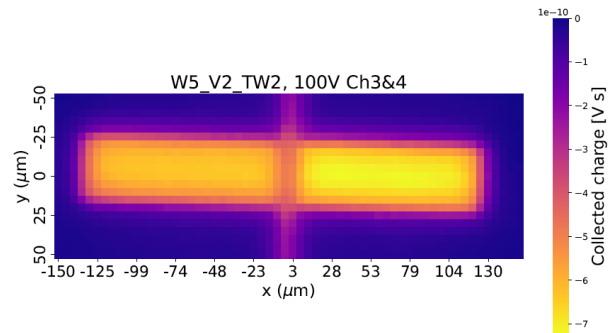
3.2 Description of the Ti-LGAD analysis program

This section provides a detailed description of the usage of the Ti-LGAD analysis program. The main script of the program is `main.py`, which serves as the entry point for the analysis. The program is designed to be modular, with different scripts handling specific tasks such as amplitude analysis, charge collection analysis, timing analysis, and plotting of results. The different parameters for the analysis can be configured in the `config.py` file. The other scripts contain the functions, often heavily centred on the Ti-LGAD measurement, needed to perform the analysis.

The software of the setup at UZH outputs a folder with multiple subfolders, containing the data files needed for the analysis, but only a few of these files are needed for the analysis. The `positions.pickle`, `measured_data.sqlite` and `parsed_from_waveforms.sqlite` files has to be added in a folder following the same structure as `LGAD_ANALYSIS/Data/SensorName/BiasVoltage/`.



(a) Microscope image of a Ti-LGAD sample with the opening in the metalisation for the TCT measurement.



(b) 2D plot of the collected charge in the area of the opening in the metalisation.

Figure 3.2: Wirebonding between the two sensors of a phase-0 module. Each pair of strips corresponds to a number printed on the edge ring.

The program has been developed for area scans around the opening of the metalisation of the Ti-LGADs samples, as shown on Figure 3.2. The typical config for the data acquisition is:

- $\text{step} = 5\text{e-}6$
- $\text{X SPAN} = 100\text{e-}6$
- $\text{Y SPAN} = 300\text{e-}6$

- LASER DAC = 500
- N TRIGGERS = 100
- Save Waveforms = False

It is important to notice that the x and y axis are usually inverted in the code. This was originating from the initial script written by and kept for the rest of the development of the code.

3.2.1 main.py

The main.py file contains the main functions used to run the analysis. Depending on the analysis the user wants to perform, the required functions must be uncommented. The different functions are grouped according to the type of analysis: amplitude analysis, charge collection analysis, timing analysis, or interpad region analysis. Two additional sections are also dedicated to debugging and the configuration of the parameterers for the noise filter.

3.2.2 config.py

This python file contains the main parameters to run the analysis. It is composed of differents classes: Paths, Filters, InterpadConfig, Subplots, PlotsConfig, and Colors. The following subsections will give a brief word on these classes.

3.2.2.1 Paths

This class is where you indicate the paths of the data the user want to analyse or the paths of the output folders. DATA_ROOT is the path of the Data folder and need to be adapted depending where the user is doing the analysis on his computer. PREFERRED_BASE_DIR is the paths of the data of a precise measurement the user wants to analyse. This is required for all the functions requireing DATAFILE (the parsed_from_waveforms.sqlite) and/or POSITIONS (the positions.pickle file) as argument. The rest of thiis class defines a method to find automaticaly the BASE_DIR if the PREFERRED_BASE_DIR does not refers to an existing folder. This method is used at the beginning of the main() function.

3.2.2.2 Filters

This class is used to define the different thresholds to cut off the noise and select the actual data. These values can be chosen by visualising them with the functions plot_amplitude(), plot_amplitude_against_t_peak() and plot_time_difference_t50().

3.2.2.3 InterpadConfig

This class defines the different parameters used for the analysis in the interpad region. The x interval on the 2D plot like the one given in Figure 3.2b defining the interpad region is given by the parameters INTERPAD_MIN and INTERPAD_MAX. The stepsize used during the measurement must also be given here. The y position where it is considered as the middle of the opening has also to be given. The INTERPAD_FRACTION defines at which fraction of the sigmoid fit in get_interpad_distance(). INTERPAD_TIMING_SCALE defines the limits of the y axis for the plots of plot_time_resolution_interpad_region_everything() (or the ones from the saved results).

3.2.2.4 Subplots

Boolean values if the subplots for the `...plot_everything()` functions have to be saved in the pdf file. This has currently not been implemented for all such subplots but can easily be done.

3.2.2.5 PlotsConfig

Configures the max values for the 2D heatmaps scale or the time interval in which the `plot_time_difference_t50()` has to focus.

3.2.2.6 Colors

Defines the list of colors used for the plots.

3.2.3 `plot_2D_maps.py`

This python file defines the functions used to plot the 2D heatmaps of amplitude, collected charge, time resolution for each or both channels. In case an analysis would be performed with one channel only, this code still works, but some lines could be commented to avoid plots of the other empty channels.

3.2.3.1 `get_sensorname_from_path()`

Returns the sensor name from the path pointing to the processed folder for this sensor.

3.2.3.2 `compute_amplitude_and_charge()`

Returns the x and y positions, a dictionary of the mean amplitude at each position in each channel and for the sum of both channels, a dictionary of the mean collected charge at each position in each channel and for the sum of both channels, and the detected active channels.

3.2.3.3 `compute_timing()`

Computes the jitter at each position, and returns a dictionary of the jitter at each position for each channel and for both channels (by taking the minimum jitter between the two channels). The Jitter is determined as the standard deviation on the time difference between the two pulses of the same trigger at 50% of the pulses. The calculated jitter is also based on the data filtered between `TIME_DIFF_MIN` and `TIME_DIFF_MAX` defined in `config.py`. The x and y positions as well as the active channels are also returned.

3.2.3.4 `plot_heatmap()`

Plots and saves the different 2D heatmaps in a pdf file. The maximum value of the z axis can be chosen from `config.py` through `AMPLITUDE_V_MAX`, `CHARGE_COLLECTION_V_MAX` and `TIMING_V_MAX`. The booleans `V_MAX_AMPLITUDE`, `V_MAX_CHARGE` and `V_MAX_TIMING` enables this manual maximum scaling while being "True", and scaled automatically while "False".

3.2.3.5 plot_2d_amplitude()

This function takes a measurement given by BASE_DIR in config.py as argument and generates a pdf of the heatmaps of the mean amplitudes recorded in each channel and a heatmap of the sum of both channels. Figure 3.3 gives an example of such heatmaps.

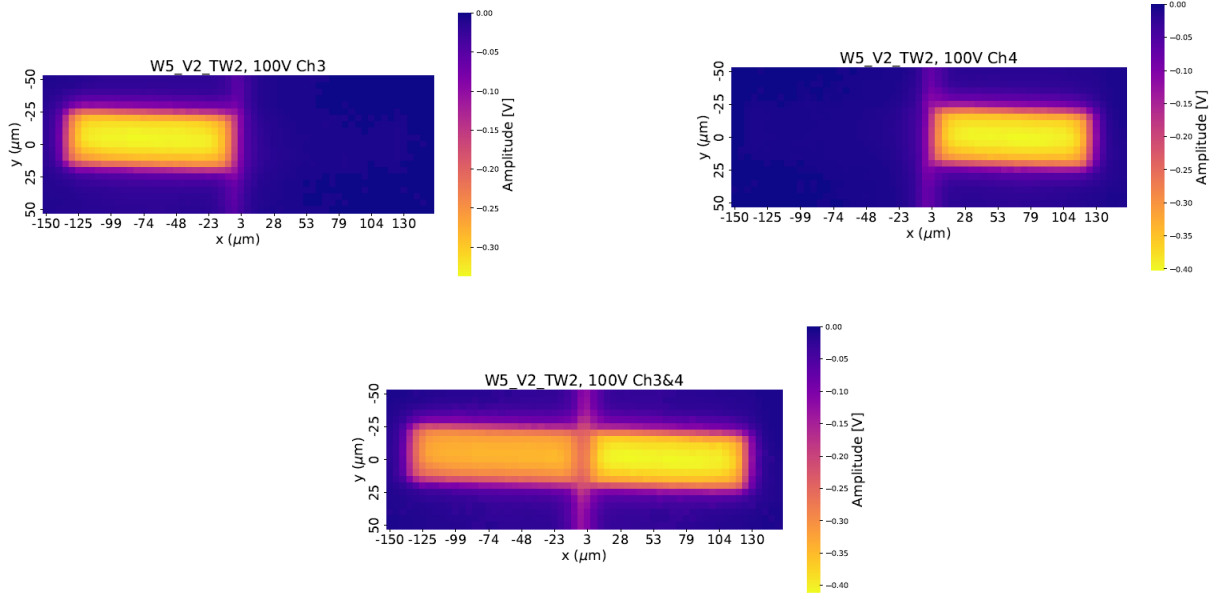


Figure 3.3: Example of amplitude heatmaps for separate and both channels with AMPLITUDE_V_MAX = 0.

3.2.3.6 plot_2d_charge()

This function takes a measurement given by BASE_DIR in config.py as argument and generates a pdf of the heatmaps of the mean collected charge recorded in each channel and a heatmap of the sum of both channels. Figure 3.4 gives an example of such heatmaps.

3.2.3.7 plot_2d_amplitude_charge_everything()

Generates a pdf file with the amplitude and collected charge heatmaps as in plot_2d_amplitude() and plot_2d_charge() for all the measurements (sensor and bias voltages) in the Data/ folder.

3.2.3.8 plot_2d_timing()

This function takes a measurement given by BASE_DIR in config.py as argument and generates a pdf of the heatmaps of the jitter in each channel and a heatmap for both channels computed by compute_timing(). An example of such heatmaps are given in Figure 3.5. Missing data is visible on the heatmaps for single channels, this comes from the fact that no jitter could be calculated at these positions.

3.2.3.9 plot_2d_timing_everything()

Generates a pdf file with the jitter heatmaps as in plot_2d_timing() for all the measurements (sensor and bias voltages) in the Data/ folder.

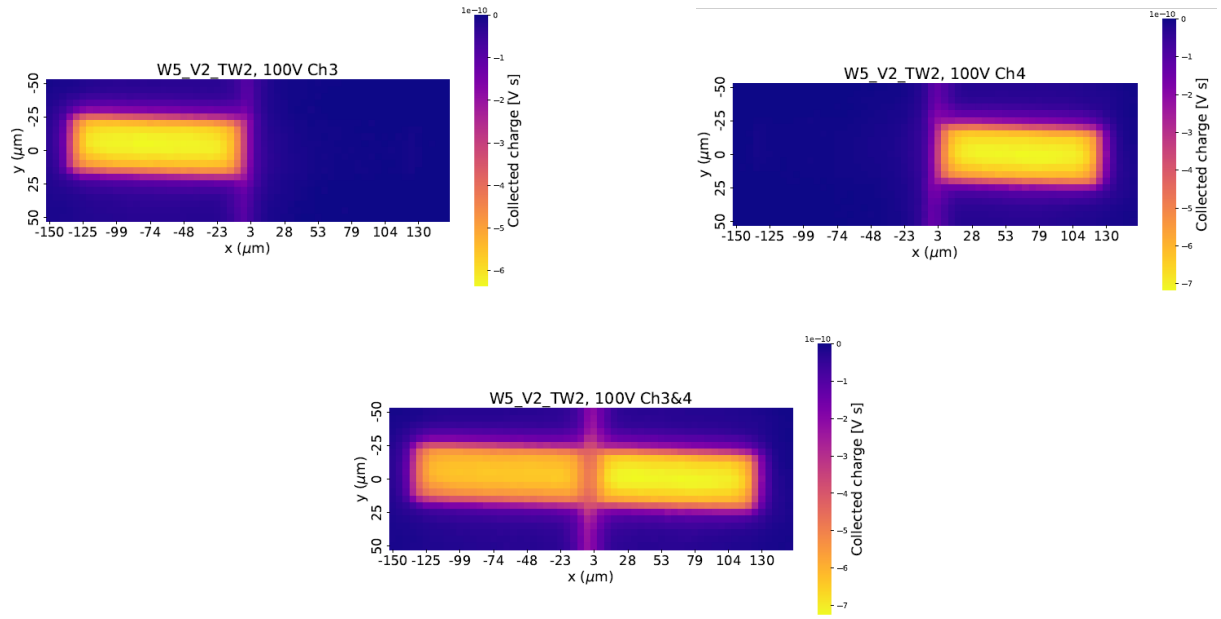


Figure 3.4: Example of collected charge heatmaps for separate and both channels with `CHARGE_COLLECTION_V_MAX = 0`.

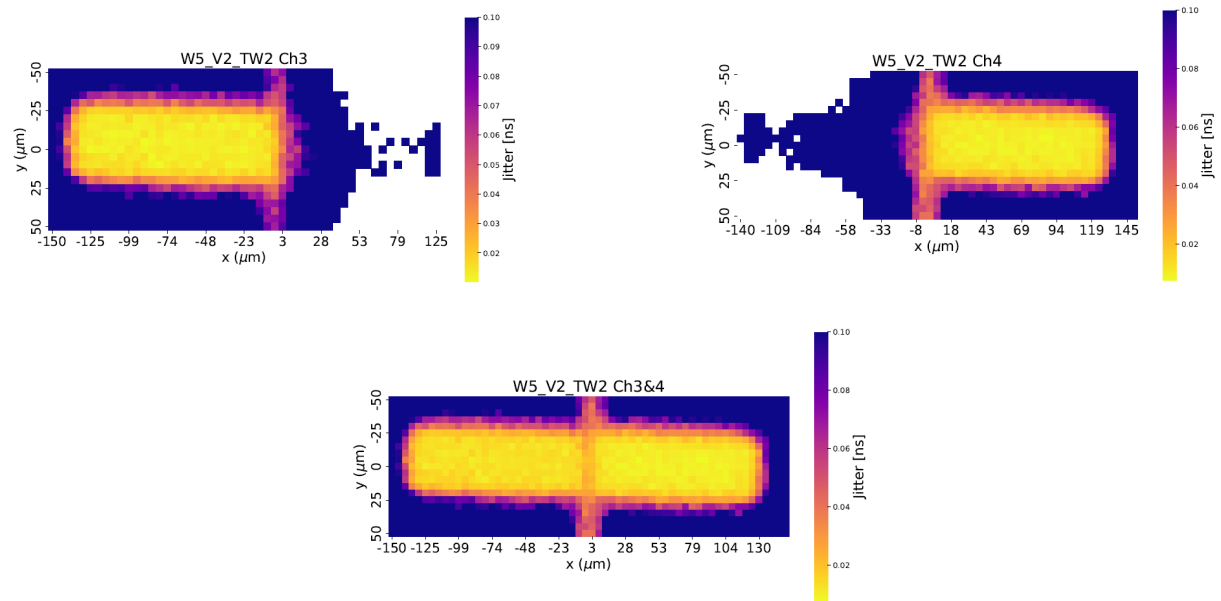


Figure 3.5: Example of time resolution heatmaps for separate and both channels with `TIMING_V_MAX = - 0.1 ns`, `TIME_DIFF_MIN = 98 ns` and `TIME_DIFF_MAX = 99.5 ns`.

3.2.4 `amplitude.py`

This python file defines the functions used for the amplitude analysis. A brief description of each function is given in the following subsections.

3.2.4.1 `plot_amplitude()`

Plots the histogram of the amplitudes recorded in each channel, after filtering the noise on the time difference between the two pulses of the same trigger and the peak time (see Sections ?? and 3.2.4.3). This enables to visualize the `AMPLITUDE_THRESHOLD` to cut the noise and an example of the output plot is given in Figure 3.6.

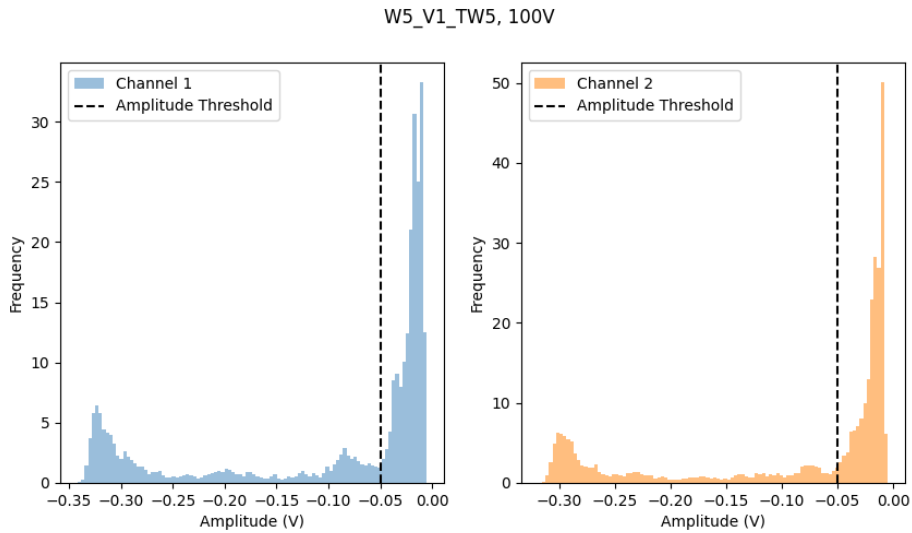


Figure 3.6: Amplitude histogram of the data after filtering on the time difference and the peak time.

3.2.4.2 `plot_noise()`

Plots the histogram of the noise recorded during the measurement. Figure 3.7 give an example of such a plot.

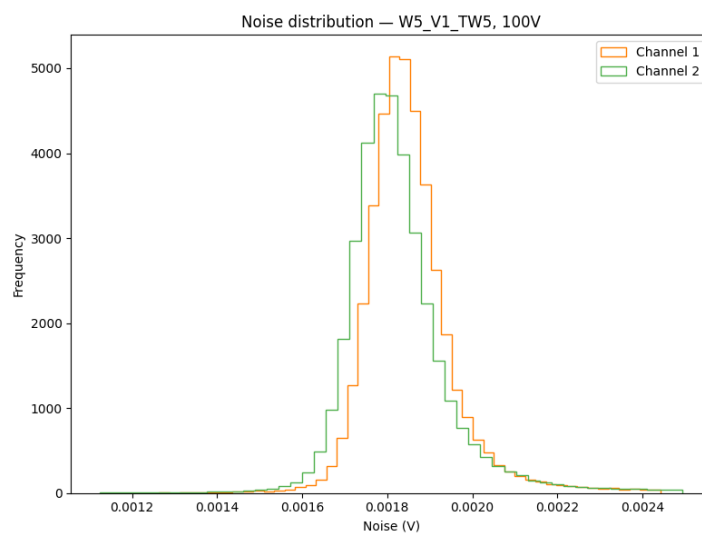


Figure 3.7: Noise histogram.

3.2.4.3 `plot_amplitude_against_t_peak()`

Plots the amplitude against the peak time calculated by default with $t_{peak} = t_{90} + 0.5 \cdot t_{over90}$. The time variable (here t_{90} and t_{over90}) can be changed by giving it as argument to the function. This function can be used to define the Filters parameters based on the peak time. An example of such a plot is given in Figure 3.8.

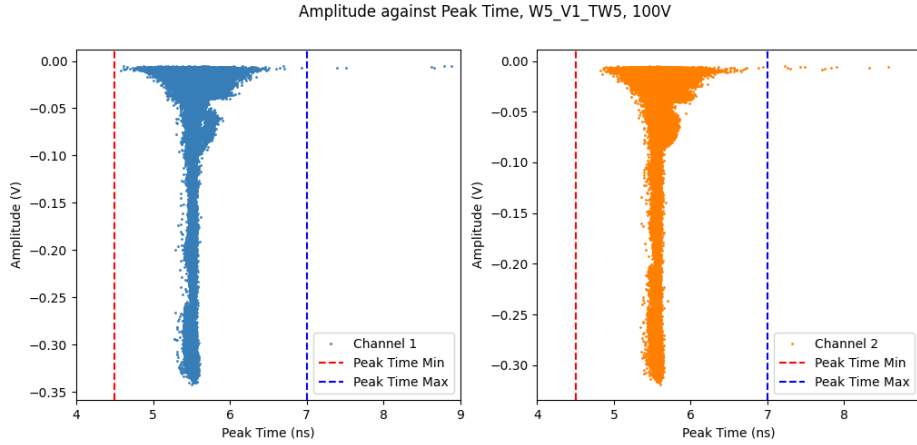


Figure 3.8: Amplitude against the peak time.

3.2.4.4 `plot_amplitude_of_one_pad()`

Plots a histogram of the amplitudes recorded in one channel, fits a gaussian curve and returns the mean amplitude and the standard deviation recorded in the channel. Figure 3.9 gives an example of such a plot (the values in the legend are rounded).

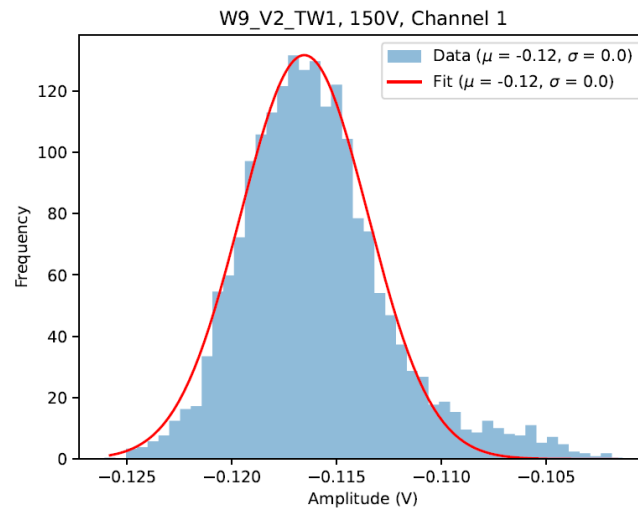


Figure 3.9: Amplitude histogram for one channel.

3.2.4.5 `plot_amplitude_everything()`

Generates a pdf file with the plots of `plot_amplitude_of_one_pad()` for each channel, bias voltage and sensor located in Data/ folder. The last plot saved in the pdf represents the mean

amplitude against the bias voltage for each channel. Figure 3.10 is an example of this plot for TCT measurements on different Ti-LGAD samples. The results of this plot are saved in .pkl files in the save directory, defined in the config.py, in order to save run time in case analysis has to be refined for one sensor. This is explained in Section 3.2.8.13.

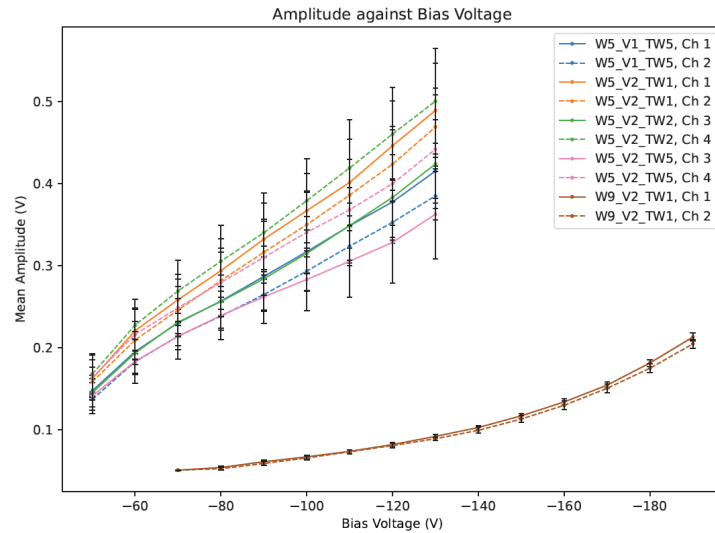


Figure 3.10: Mean amplitude against bias voltage for each channel.

3.2.4.6 plot_2D_separate()

This function outputs the plots on Figure 3.11. The right hand side of the Figure represents the 2D plot of the amplitude recorded in each channel and the left hand side represents a projection of this amplitude on the y axis. This is a not modified function of the original code and must be adapted to correct the plots on the left hand side. In fact, this projection is calculated with avrages on the whole data in each y position, taking the noise and the 0 amplitude data into account, resulting in a underestimation of the real amplitude abserved in each channel and un large standard deviation (resulting in large error bars).

3.2.4.7 project_onto_y_one_channel_amplitude()

Returns a dictionary with the amplitude projection of in the interpad region defined in the config.py. This function was created to compute the interpad distance with the amplitude (see Section 3.2.7). The fact that the name of the function refers to a projection along the y axis and actually project along the x axis on Figure 3.3 is because there is an inversion between the axis compared to the actual data outputted by the measurement and can lead to some confusions.

3.2.4.8 plot_amplitude_along_y_axis()

This function plots the projection of the amplitude and the collected charge in each channel for one selected y value in the middle of the opening (defined in config.py) along the y axis of Figures ?? and 3.2b. Figure 3.12 represents an output of this function.

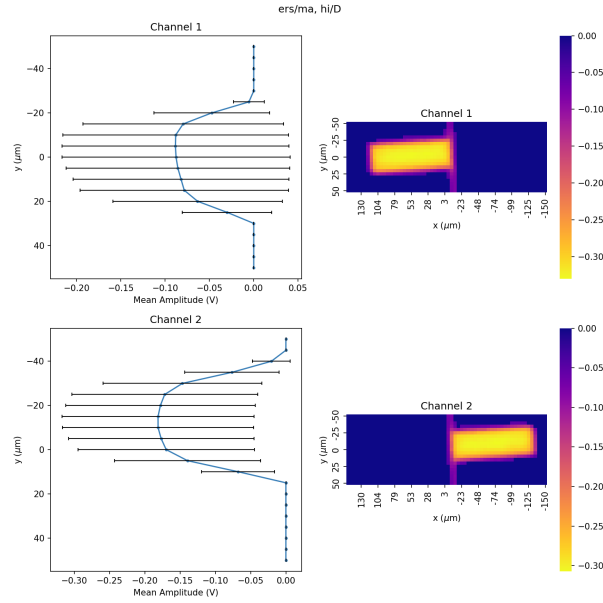


Figure 3.11: Output of `plot_2D_separate()`.

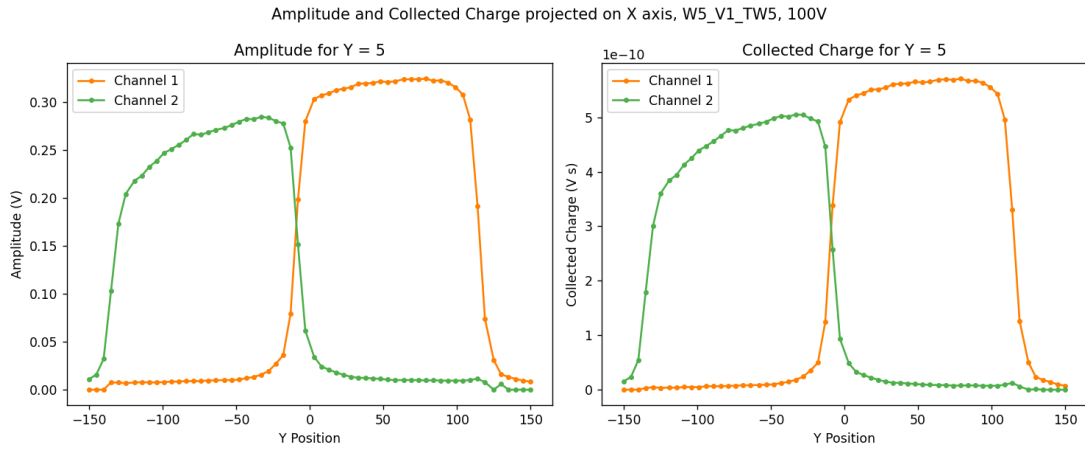


Figure 3.12: Projection of the normalised amplitude and the collected charge on x for a given y of the 2D plots on Figures ?? and 3.2b.

3.2.5 `charge_collection.py`

This python file defines the functions related to the charge collection analysis of the measurement.

3.2.5.1 `plot_collected_charge()`

Plots the histogram of the collected charge recorded in each channel, after filtering the noise on the time difference between the two pulses of the same trigger, the peak time and the amplitude (see Sections ??, 3.2.4.3 and 3.2.4.1). An example of the output of this function is given in Figure 3.13.

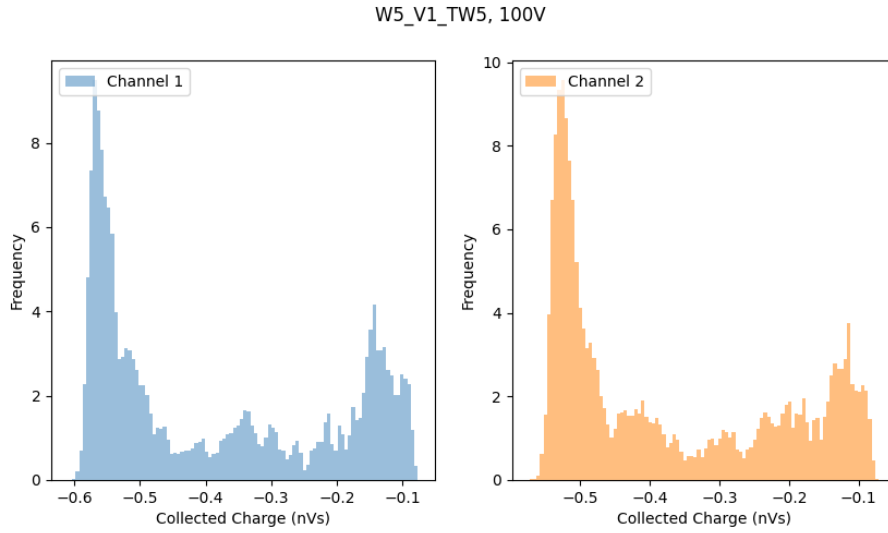


Figure 3.13: Collected charge histogram after filtering the noise on the time difference between the two pulses of the same trigger, the peak time and the amplitude.

3.2.5.2 `plot_collected_charge_of_one_pad()`

Plots a histogram of the collected charge recorded in one channel, fits a gaussian curve and returns the mean collected charge and the standard deviation in the channel. Figure 3.14 gives an example of such a plot (the values in the legend are rounded).

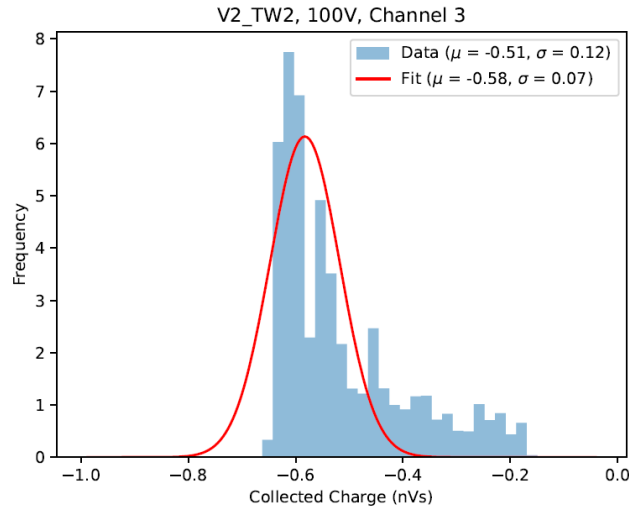


Figure 3.14: Collected charge histogram for one channel.

The gaussian fit can be discussed since the distribution on the histogram is not completely symmetric and correct. To describe obtain a physical fit, it should follow the convolution of the probability distribution to have M photons in the laser pulse, a poissonian distribution (probability density to absorb N photons and generate a electron-hole pair), the gain distribution of the LGAD (probability density to generate the charge Q from a electron-hole generation), the gain distribution of the amplifier and the the random electronic noise distribution.

3.2.5.3 plot_collected_charge_everything()

Generates a pdf file with the plots of `plot_collected_charge_of_one_pad()` for each channel, bias voltage and sensor located in `Data/` folder. The last plot saved in the pdf represents the mean collected charge against the bias voltage for each channel. Figure 3.15 is an example of this plot for TCT measurements on different Ti-LGAD samples.

The results of this plot are saved in `.pkl` files in the `save` directory, defined in the `config.py`, in order to save run time in case analysis has to be refined for one sensor. This is explained in Section 3.2.8.13.

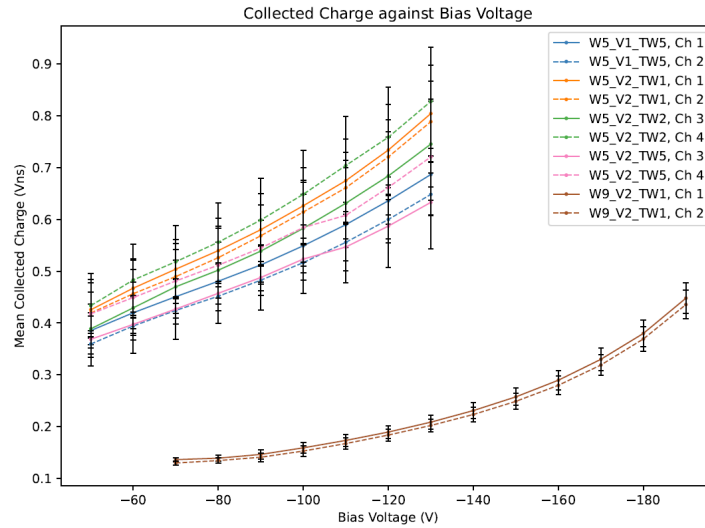


Figure 3.15: Mean collected charge against bias voltage for each channel.

3.2.5.4 project_onto_y_one_channel_charge()

Returns a dictionary with the collected charge projection in the interpad region defined in the `config.py`. This function was created to compute the interpad distance with the collected charge (see Section 3.2.7). The fact that the name of the function refers to a projection along the y axis and actually project along the x axis on Figure 3.4 is because there is an inversion between the axis compared to the actual data outputted by the measurement and can lead to some confusions.

3.2.6 timing.py

This python file defines the functions related to the timing analysis of the measurement.

3.2.6.1 plot_time_difference_t50()

Plots the time differences between the two pulses of the same trigger at calculated at 50% of the pulses for each separate channel. The parameters of the plot are defined in the `PlotsConfig` class of `config.py`, such as the minimum and maximum of the abscise and the number of bins for the histogram. Figure 3.16 represents such a plot and can be used to define `TIME_DIFF_MIN` and `TIME_DIFF_MAX` in `config.py` for the noise filtering.

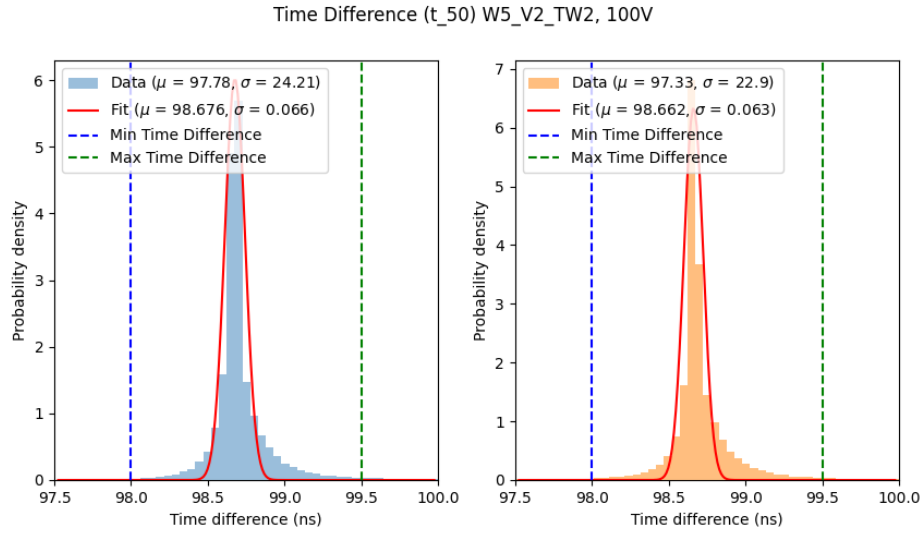


Figure 3.16: Measured time difference histogram between the two pulses of the same trigger at 50% of the pulses for each separate channel.

3.2.6.2 `plot_time_resolution_of_one_pad()`

Plots the histogram of the time differences between the two pulses of the same trigger at 50% of the pulses measured in one channel. A gaussian is fitted on this distribution to determine the jitter (standard deviation), proportional to the time resolution. Figure 3.17 is given as example of such a plot.

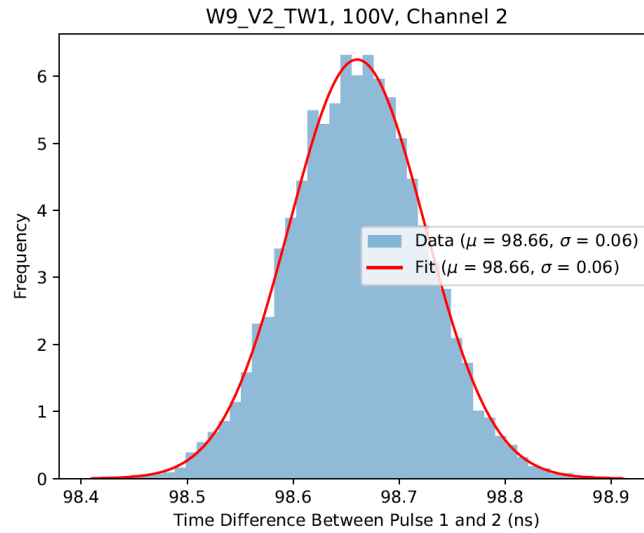


Figure 3.17: Measured time difference histogram in one channel.

3.2.6.3 `plot_time_resolution_everything()`

This function generates a pdf file with the plots given by `plot_time_resolution_of_one_pad()` for each measurement in the Data/ folder. This function also plots the jitter as a function of the bias voltage for each measured sample in each separate channels, as depicted in Figure 3.18.

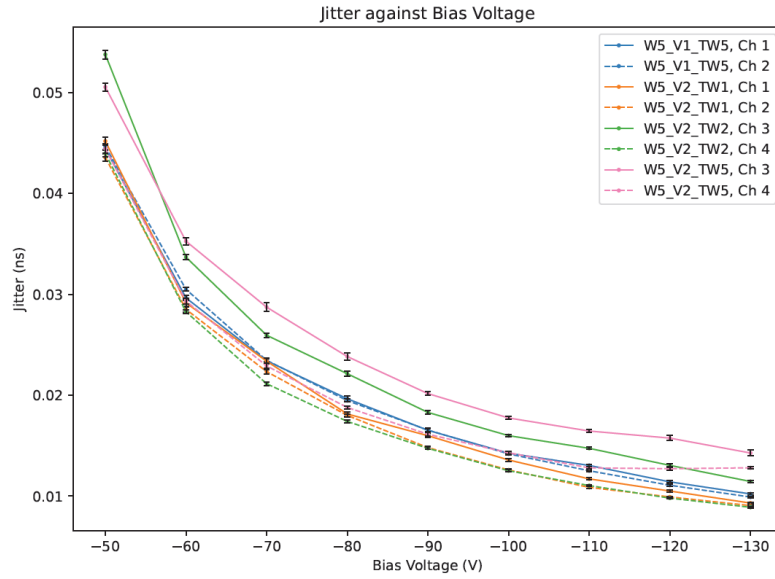


Figure 3.18: Jitter as a function of the bias voltage.

3.2.6.4 `plot_time_difference_histogram()`

This function has not been significantly modified from the initial version of the code. It is not used elsewhere in the code and is currently not working, during an average and standard deviation calculation resulting in the error “`ValueError: array must not contain infs or NaNs`”. The goal of this function has been identified as the plot of a histogram of the time differences between the two pulses of the same trigger for a given y position of the plot of Figure 3.5 for both channels, with the objective to fit a gaussian to extract the jitter.

3.2.7 `interpad.py`

This python file contains the definitions of the functions used for the analysis between the two pixels, and in particular, at the trench region.

3.2.7.1 `sigmoid()`

This function defines the sigmoid function used for fits in `get_interpad_distance()`. It is defined as:

$$y(x) = \frac{L}{1 + e^{-k(x-x_0)}} + b \quad (3.1)$$

with L a scaling factor, positive for a rising sigmoid and negative for a decreasing sigmoid, k the slope (steepness) of the transition, x_0 the position of the inflection point of the curve, and b a vertical offset.

3.2.7.2 `get_interpad_distance()`

This function generates a plot similar than the one on Figure 3.19 and returns the Inter-Pad Distance (IPD) as well as its error. The plot represents the normalized collected charge in each channel projected on the axis perpendicular to the trench for a given measurement. A sigmoid function is fitted on the collected charge in each channel. The IPD is calculated as the distance at which the sigmoids reach 90% of its maximum between the two channels. This

parameter is used to evaluate the range of the no-gain region between the pixels and to study the different trench designs. The green line represents the sum of the normalized collected charge in both channels. It is also possible to compute the IPD with amplitude projections by commenting and uncommenting some lines at the beginning of the function.

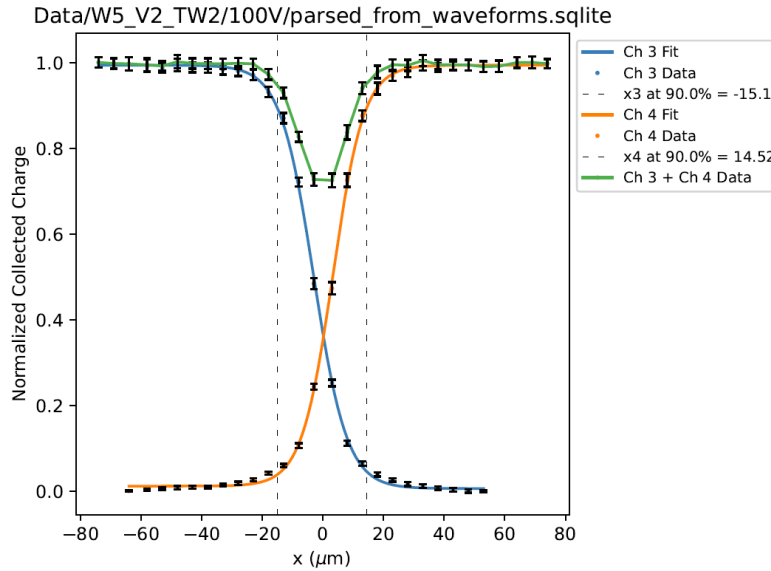


Figure 3.19: Plot representing how the IPD is defined and calculated.

3.2.7.3 plot_interpad_distance_against_bias_voltage_v2()

This function generates a pdf file with all the output plots of `get_interpad_distance()` for each measurement in the Data/ folder. Furthermore, it generates a plot representing the IPD as a function of the bias voltage for each sample (as shown on Figure 3.20). The results of this plot are saved in .pkl files in the save directory, defined in the config.py, in order to save run time in case analysis has to be refined for one sensor. This is explained in Section 3.2.8.13.

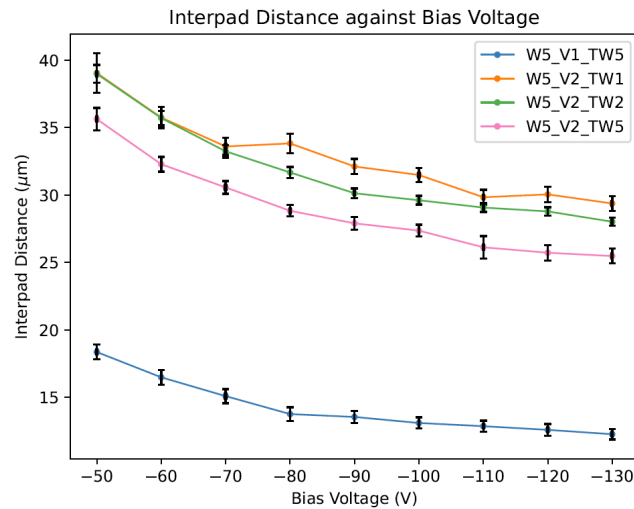


Figure 3.20: IPD as a function of the bias voltage for different samples.

3.2.7.4 `plot_time_resolution_interpad_region()`

This function is not used anymore but was used to make a similar plot as Figure 3.22b. This difference with `plot_time_resolution_interpad_region_v2()` reside in the selection of the data to compute the jitter. This function was computing the jitter with all the data that was passing through the noise filter defined in the `config.py`. This led to two drawbacks: the filter was too strict for some data at the trench resulting in missing results at low bias voltages, and the projection on the x axis on the plot of

`plot_time_resolution_interpad_region_everything()` was not robust to a tilted sample (trench not completely perpendicular to the x axis). An example of the output with missing data at the trench is given on Figure 3.21.

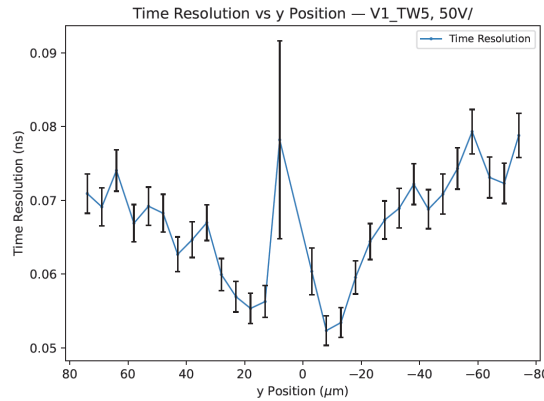


Figure 3.21: Example of an output plot with missing data at the trench.

3.2.7.5 `freedman_diaconis_bins()`

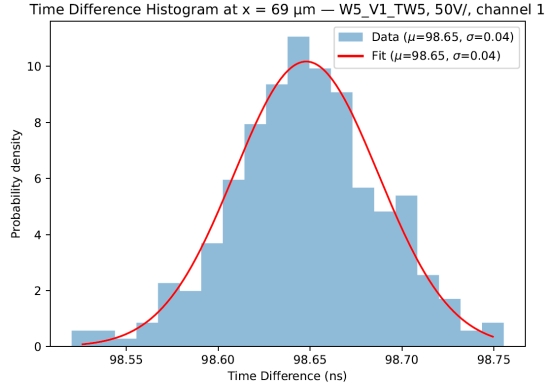
This function defines the Freedman-Diaconis rule to select the width of the bins of histograms. This function has been suggested by ChatGPT for the histograms in `plot_time_resolution_interpad_region_v2()`.

3.2.7.6 `plot_time_resolution_interpad_region_v2()`

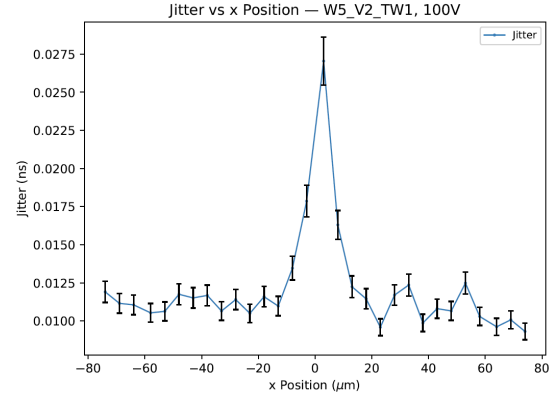
This function takes a measurement and a pdf file as argument. The jitter is calculated for each x coordinate and for each channel in the inter-pad region with the data at $Y_POSITION_MID_PAD \pm Y_STEP$, defined in the `config.py`. This is performed in order to calculate the jitter on more statistics and can be modified with list `Y_STEPS` within `plot_time_resolution_interpad_region_v2()`. It results in a plot similar as the one in Figure 3.22a. Once the jitter is calculated along the whole inter-pad region, it plots the jitter as a function of the x distance, as shown on Figure 3.22b. The plotted jitter corresponds to the minimum jitter between the two channels. All the plots are saved in the pdf file.

3.2.7.7 `plot_time_resolution_interpad_region_everything()`

This function generates a pdf file with the same plots as in Figure 3.22 for each measurement in the `Data/` folder and generates a plot similar to the one in Figure 3.23 for each sensor, providing information on the evolution of the jitter at the trench of the Ti-LGAD with the bias voltage.



(a) Time difference histogram with the data at a given x coordinate in the inter-pad region and $Y_POSITION_MID_PAD \pm Y_STEP$.



(b) Jitter projection in the inter-pad region.

Figure 3.22: Example of output plots of `plot_time_resolution_interpad_region_v2()`.

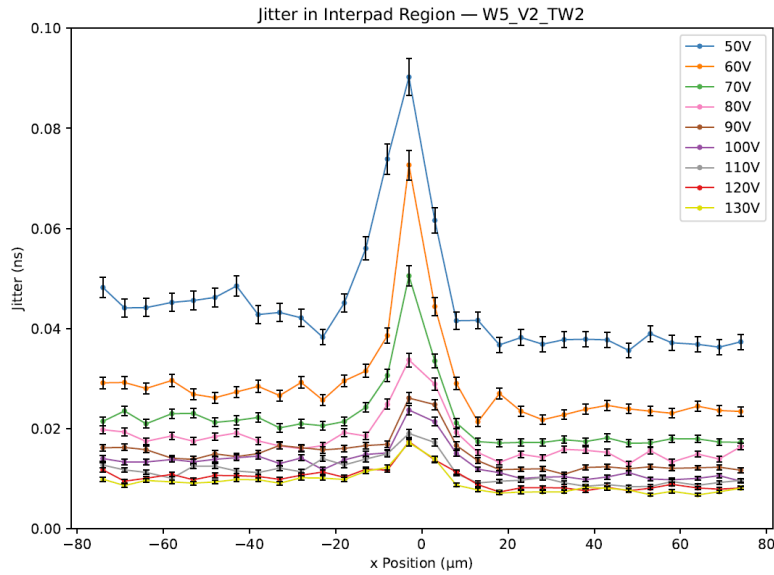


Figure 3.23: Jitter in the inter-pad region as a function of the bias voltage.

3.2.8 data_manager.py

This Python file defines the functions used to process and select the data, acting as a toolbox.

3.2.8.1 gaussian()

Defines the gaussian function for gaussian fits as:

$$y(x) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (3.2)$$

3.2.8.2 query_dataset()

Takes a `parsed_from_waveforms.sqlite` file of a measurement as argument and returns the number of (x,y) positions that were measured, the number of triggers at each position

and the number of channels that were recorded with the oscilloscope.

3.2.8.3 `get_positions()`

Gets a `positions.pickle` file as argument, put the origin of the (x,y) frame at the center of the measurement and returns the (x,y) coordinates in μm for each position. These coordinates are the actual measured coordinates and were inverted in comparison to the 2D plots given in the other Sections.

3.2.8.4 `get_bias_voltage()`

Gets a `parsed_from_waveforms.sqlite` file as argument and returns the applied bias voltage with its uncertainty. This function originates from the original code and has not been tested.

3.2.8.5 `get_channel_amplitude()`

Gets a measurement (a `parsed_from_waveforms.sqlite` file) as argument and returns the amplitudes recorded at each position averaged on the different triggers. These amplitudes are filtered with the amplitude threshold, the time differences between the two pulses of the same trigger and the peak time. This function has not been implemented elsewhere in the code and originates from the initial code.

3.2.8.6 `determine_active_channels()`

Returns the tuples (1,2) or (3,4) depending on which channels are active based on the mean amplitudes recorded. This function must be modified in case other combinations of recorded channels are done (e.g. (1,4)) or if more than two channels must be analyzed.

3.2.8.7 `get_pad_positions()`

Returns the list of positions corresponding to valid signals in a given channel, i.e. the positions where the data pass the filters defined in `config.py`.

3.2.8.8 `plot_pad_positions()`

Plots the positions where valid signal has been detected by `get_pad_positions()` in each channels for a given measurement defined by the `BASE_DIR` in `config.py`. Figure 3.24 is a representation of such a plot and the (x,y) coordinates correspond to the actual measured coordinates returned by `get_positions()`.

3.2.8.9 `plot_pad_position_everything()`

This function generates a pdf file with plots of the positions of the filtered data is considered as signal for all the measurements in the `Data/` folder. It plots the pad positions for separate channels and for both of them, as shown on Figure 3.25 for one measurement. The last page of the pdf contains the values of the parameters used for the filter in `config.py`.

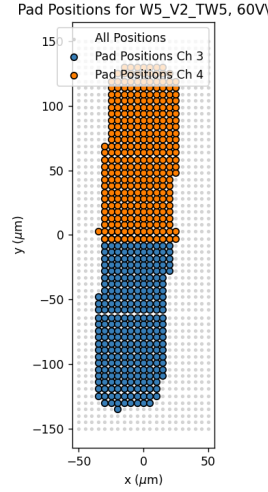


Figure 3.24: Example of output plot by `plot_pad_positions()`.

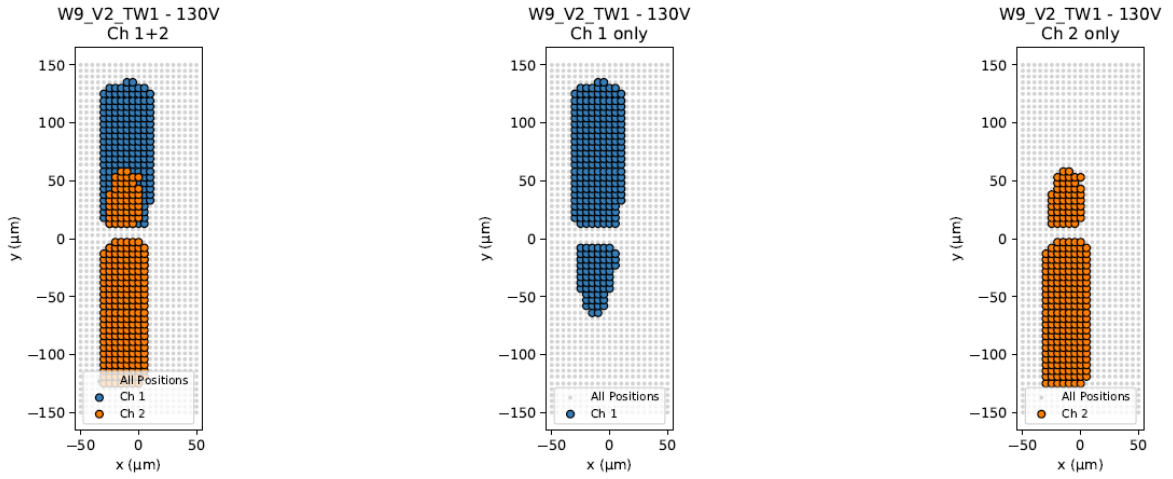


Figure 3.25: Example of an output plot by `plot_pad_positions_everything()` for one measurement. Cross-talk can be observed between the channels in this example.

3.2.8.10 `get_sensor_strip_positions()`

Returns the positions used for the IPD calculations. First, it filters out the data and selects the region of interest (the inter-pas region). The (x,y) frame used in this function is the measured frame saved in the database. It then evaluates amplitude projections along x for bins located near the edges in the y-direction and selects the edge exhibiting the lowest mean amplitude (knowing the amplitudes are negative values). Based on the minimum of this projection and its associated uncertainty, the code identifies the corresponding valid x positions. These are finally combined with a selected range of y values to determine the indices defining the strip positions.

The `POINTS_FROM_EDGE_STRIP_POSITION` parameter defines the number of points in the y-direction not selected from the edge on the side where the signal is expected to be weak (e.g. on the positive side for the blue curve and the negative side for the orange curve in Fig. 3.19). This can be used when there is some cross-talk between pads in order to remove part of the data and obtain a more reliable sigmoid fit.

This method to select the data to compute the IPD is still the original one and may have some limitations. Indeed, a mismatch between the selected data between the two channels

for the x coordinate can lead to selected data as depicted on Figure 3.26. This could lead to unexpected IPD results if the sensor is slightly tilted. It would be interesting to force the code to select matching data between the two channels.

3.2.8.11 `plot_sensor_strip_positions()`

Plots the strip positions selected by `get_sensor_strip_positions()`, as shown on Figure 3.26.

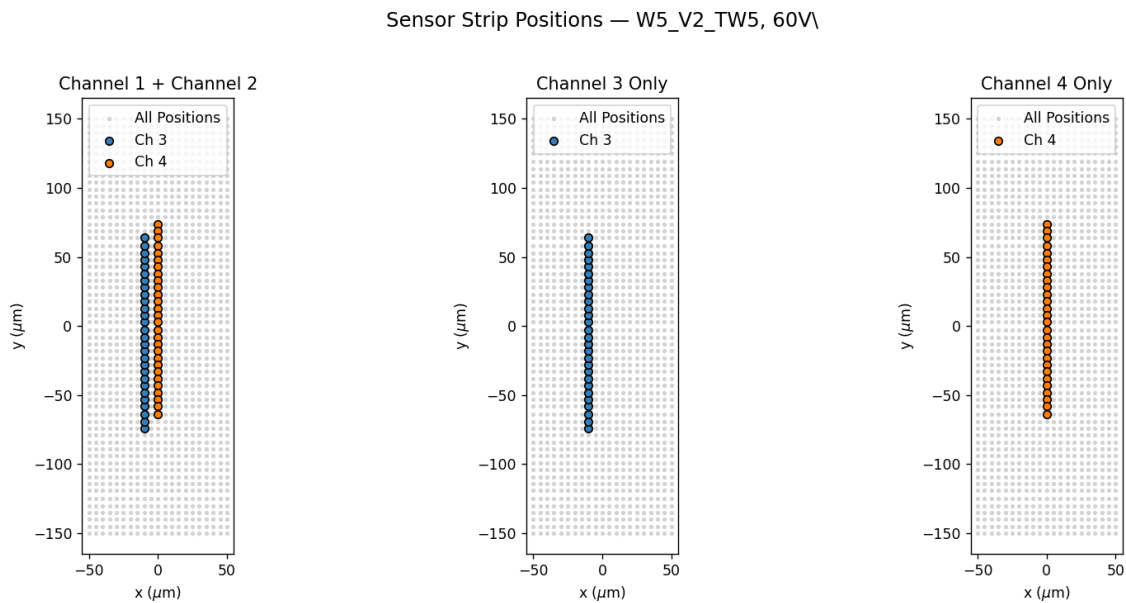


Figure 3.26: Example of an output plot by `plot_sensor_strip_positions()` for one measurement. It can be observed that the selected data can mismatch between the two channels.

3.2.8.12 `detect_type()`

Detects the type of an object (data) given as an argument. It returns:

- a dictionary if the object contains results from the amplitude, charge collection, or timing analysis;
- a tuple of length 3 if the object contains the results of the timing analysis in the inter-pad region;
- a tuple of length 4 if the object contains the results of the IPD analysis in the inter-pad region.

This function has been written while writing `save_results()` and its functionality is not completely used in the rest of the code.

3.2.8.13 `save_results()`

This function takes the type of the analysis for which the results have to be saved and the actual data to same. It can save the final results of `plot_amplitude_everything()`, `plot_collected_charge_everything()`, `plot_time_resolution_everything()`,

`plot_interpad_distance_against_bias_voltage_v2()` and `plot_time_resolution_interpad_region_everything()` in pickle files. It was created in order to reload results without running the analysis for all sensors again (which is time consuming). The structure of the save directory is given in Figure 3.27.

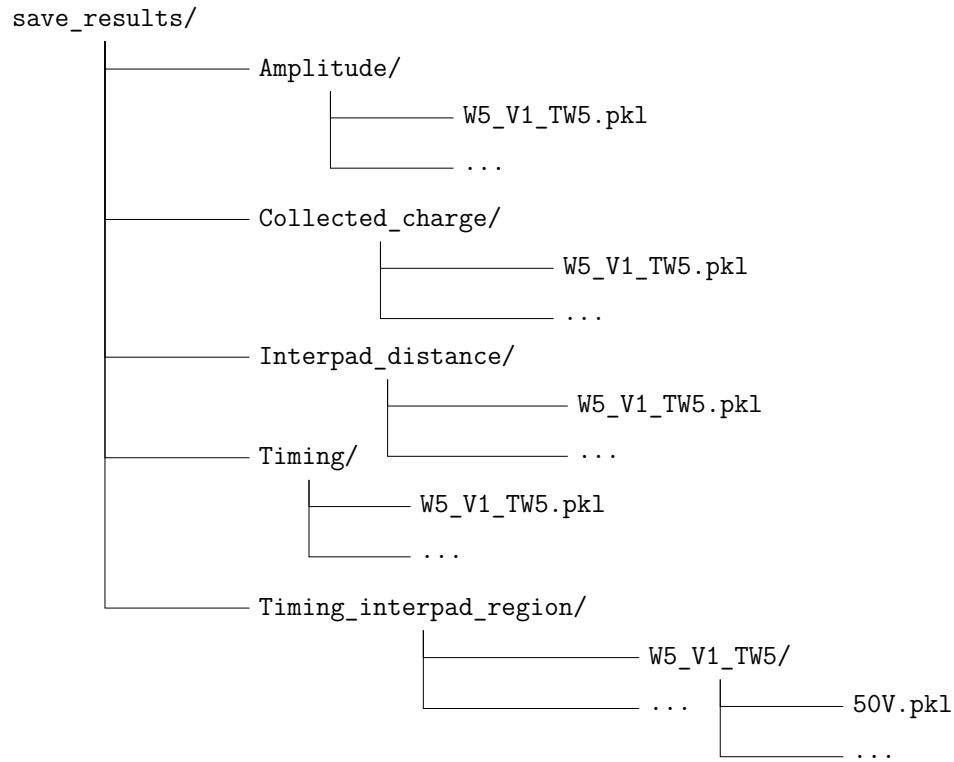


Figure 3.27: Structure of the save directory.

3.2.8.14 `load_results()`

This function takes the path to a saved pickle file and returns the stored data along with its type.

3.2.8.15 `merge_saved_results()`

This function was intended to merge results from two saved pickle files but was left unfinished and does not work, as it was not required for the analysis performed.

3.2.9 `plot_saved_results.py`

This python file defines the functions that plot the data saved in pickle files.

3.2.9.1 `plot_saved_results()`

This function takes a string as an argument indicating which data should be loaded and plotted, depending on the selected type of analysis. It generates a pdf file containing the final plots of all results for the chosen analysis.

Chapter 4

Results

Chapter 5

Conclusion

Bibliography

- [1] M. Senger. Development of Precise Timing and Position Detectors Towards 4D Tracking. PhD thesis. University of Zurich, (2024).